

Security Policy verification: how and why

1. compiling this document

This document can be compiled with the following command: `groff -ms report.ms -Tpdf > report.pdf`

2. Introduction

The LIS project requires verification of different security policies, all of which have in common that they can be verified by scanning the whole history of each terminating symbolic state. The idea can be implemented by keeping a state for each policy and updating the state at each service invocation. This implementation has a drawback and an advantage:

- drawback: Adding a state for each policy will make the symbolic state heavier, consuming more memory
- advantage: if the policy goes into a bad state, then we can say that the policy is violated and stop early, reducing the time spent during symbolic execution.

If the drawback has more weight than the advantage, the solution can easily be applied in the initial strategy of scanning the whole history at the end of the symbolic execution, given that the state of a policy is updated using an element of the history.

3. From Security Policy to Policy Checker

In the language that is used for service contracts there are three types of security policies:

3.1. Qos Field Operation

Does an aggregate operation (sum, min, max, avg) on a Quality of Service vector field, then compares (less than, greater than, ...) the result with an integer. If the comparison is not satisfied and cannot be satisfied with any successive service invocations, then the policy is violated.

3.2. Regex

Checks if a prefix of the history matches a regex. If it finds a match, then the policy is violated. The regex works only on the names of services invoked

3.3. Sort (Ascending, Descending)

Given the name of a QoS vector field, the policy checks that the symbolic value of this field never decreases (Ascending) or never increases (Descending)

4. Parsing and mapping

These policies are parsed and then each one is mapped to the corresponding Policy Checker, which just contains the state of the policy and all the information needed to compute the next state when paired with the service invocation. There exists an option in the service contract definition language to say that a policy is checked for each subsequence of the history resulting from a `groupBy` operation: the `groupBy` takes the name of a parameter and groups all the invocations that have the same argument assigned to that parameter. If a service does not have that parameter, it will not appear in any group, as expected, so it will be ignored by the security policy declared in this way. This kind of policies will be called grouped policies, while the other policies will be called ungrouped. All three types of policies defined before can be declared grouped or ungrouped.

4.1. On the AVG aggregator

In the policy that does the QoS field aggregation and then a comparison, it is possible to set `aggregate = avg`. The Policy Checker state here can't be just a symbolic integer, because this will not be enough information to compute the average. This means that the state of the policy will be a pair (`symbolic int, int`), so that the symbolic integer contains the sum and the integer will be just a counter of service invocations seen so far (in the whole history if ungrouped, in the group if grouped).

5. Technical details

Those people in the team that write the security policies should only care about the type of Policy Checker they are working with, without worrying about if the policy is grouped or ungrouped: the `map_state` function was written to aid them by making the implementation only a matter of writing a function `f` from state to state. The `map_state` function expects the following parameters: `map_state(initial, f, call, service, current_state)`

`initial`

The initial state of the Policy Checker (example: for Qos field Operation with `aggregate = avg` the initial state is (`symbolic 0, 0`), while for sum it is just 0.

`f` The function to map the current state to the next one

`call` The service invocation: it contains the service name, the argument list, the map of Qos vector fields (map name -> field). Getting one argument by name of service parameters is possible, the implementation of the function `map_state` has one example on how to do it.

`service`

The service that has been invoked, with all the parameters name, pre and post, etc.

`current_state`

The current state of the policy Checker